



College of Computing and Informatics

Computer Science department

Programming languages and paradigms

Final report (HerbLang)

Student	ID	Section
Amira Khezzar	U23104100	61
Hasna Imad Eldin Abdul Hay	U23100997	61
FATMA A. S. ALKHODARI	U22104519	62
Mona M. M. Mansour	U22104988	62
Fatma ibrahim khammas	U23101181	62

Instructor: Dr. Imad Afyouni

Contents

1. Concept Overview	2
2. Syntax Design	3
3. Motivation	3
4. Paradigm & Implementation	3
5. Semantics	4
5.1 General Evaluation Rule	4
5.2 Mathematical Semantics	4
5.3 Operational / Recursive Semantics	4
5.4 Entity Meaning (Conceptual Semantics)	5
6. Grammar (CFG)	5
6.1 Introduction	5
6.2 Context-Free Grammar (CFG)	5
7. Lexical Design:	8
7.1 Token Types	9
7.2 Tokenization Algorithm	9
7.3 Lexer Examples	10
seed 5 +++> 3 flower	10
8. Parser implementation	11
8.1 Design — Recursive Descent	11
8.2 Parsing Arithmetic Expressions	11
8.3 Parsing Function Calls	12
8.4 Error Reporting	12
9. Abstract Syntax Tree (AST):	13
9.1 AST Node Structure	13
Arithmetic Node	13
Function Node	13
9.2 AST Examples	13
9.3 Information Preserved in the AST	14
10. Interpreter:	14
10.1 Pipeline Overview	14
10.2 Evaluating Arithmetic Nodes	15
10.3 Worked Arithmetic Evaluations	16

10.4 Evaluating Function Nodes	16
10.5 Worked Function Evaluations	17
11. Built-in- functions	18
12. Extended Examples	18
12.1 Arithmetic Expressions	19
12.2 Function Calls.....	19
13. Limitations	20
13.1 Future Work	20

1. Concept Overview

The proposed language is a nature inspired symbolic programming language that models computation through natural processes such as growth, flow, decay, and division. Rather than relying solely on conventional mathematical notation, the language integrates entities to give meaning and context to operations. These entities, such as seed, grow, cut, and split, conceptually represent different types of transformations applied to values.

A central feature of the language is its use of repetition-based operators, where the number of times an operator symbol appears determines the number of times the operation is applied. This approach introduces a visually intuitive and scalable mechanism for expressing computation, aligning symbolic representation with the magnitude of transformation.

In addition to arithmetic expressions, the language supports higher-level functional constructs that model natural processes, extending its expressive capabilities beyond basic operations. These functions follow a consistent conceptual framework aligned with the overall nature-based design of the language.

2. Syntax Design

The syntax of the language reflects its nature-based conceptual model while maintaining clarity and consistency. Each arithmetic expression follows a structured pattern in which an initial value is transformed through a repeated operation using a secondary value.

The general form of an expression is:

<entity><number> <operator sequence> > <number><entity>

In this structure, the first entity-number pair represents the initial value, while the operator sequence like +, -, *, or / determines both the type and the number of times the operation is applied. The arrow symbol (>) separates the initial value from the operand.

The second number, associated with an entity, serves as the value used in repeated operation. The expression is evaluated by applying the operation iteratively based on the number of operator symbols.

3. Motivation

Traditional programming languages use abstract symbols like +, -, *, and / that carry no inherent meaning. A beginner seeing $x = x + 1$ sees no connection between the symbol + and the concept of growth or increase. This forces learners to memorize syntax without understanding why operations work the way they do.

Our language solves this by embedding arithmetic within nature-based metaphors. When a user writes `seed5 +++> 3flower`, they are not just adding numbers — they are simulating growth. A seed (value 5) receives sunlight (the + operator) three times, producing a flower (value 14). Similarly, `cut10 ---> 2leaf` models pruning or decay, where a value is reduced through repeated subtraction. These metaphors make the code self-explanatory.

Another limitation of traditional languages is that repetition must be expressed explicitly using loops or by writing the same operation multiple times. Our language solves this by making the number of operator symbols directly control the repetition count. Writing `+++` is visually intuitive: more plus signs mean more addition.

In summary, our language is motivated by three core goals:

1. **Semantic alignment:** Operations should mean something beyond just math
2. **Repetition as syntax:** Repeated operators should express repeated operations
3. **Engagement through metaphor:** Nature based entities make programming intuitive and fun

4. Paradigm & Implementation

Our language is built using functional programming paradigm even though we implement it in Python.

1. **Immutability:** Once a value is defined, it never changes. A seed (5) does not become a different seed; rather, a new flower (14) is produced. No existing values are modified; only new values are created.
2. **Pure Functions:** Every operation produces the same output for the same input with no side effects. `seed5 +++> 3flower` always evaluates to 14, regardless of when or where it is called.
3. **Recursion over Loops:** Traditional loops rely on mutable state. Instead, we use recursion to implement repetition. Adding a number of multiple times is implemented as a recursive function that calls itself until a counter reaches zero.
4. **First Class Functions:** Functions can be passed as arguments and returned as values, allowing higher-order operations.
5. **Referential Transparency:** Any expression can be replaced with its value without changing the program's behavior.

The language is implemented in Python 3, chosen for its readability, minimal boilerplate, support for recursion, and fast prototyping.

The implementation consists of four components:

1. **Lexer** (detailed in Lexical Design): converts source code into tokens
2. **Parser** (detailed in Grammar section): builds an AST from tokens
3. **Interpreter** (detailed in Semantics): evaluates the AST using recursion
4. **REPL** (detailed in Implementation): handles user interaction

5. Semantics

5.1 General Evaluation Rule

An expression of the form:

$\langle \text{entity}_1 \rangle \langle n_1 \rangle \langle \text{operator_seq} \rangle \langle n_2 \rangle \langle \text{entity}_2 \rangle$

is evaluated as follows:

- n_1 represents the initial value
- n_2 represents the operand value
- The operator (+, -, *, /) defines the type of operation
- The number of operator symbols (**k**) determines how many times the operation is applied

Thus, the operation is applied k times to n_1 using n_2 .

5.2 Mathematical Semantics

Addition: $\llbracket n_1 +^k n_2 \rrbracket = n_1 + (k \times n_2)$

Subtraction: $\llbracket n_1 -^k n_2 \rrbracket = n_1 - (k \times n_2)$

Multiplication: $\llbracket n_1 *^k n_2 \rrbracket = n_1 \times (n_2^k)$

Division: $\llbracket n_1 /^k n_2 \rrbracket = n_1 \div (n_2^k)$

5.3 Operational / Recursive Semantics

Example (Addition):

```
eval(n, k):  
if k == 0:  
    return n  
else:  
    return eval(n + n2, k - 1)
```

5.4 Entity Meaning (Conceptual Semantics)

- Seed --> Initial value
- Grow --> Increasing transformation
- Cut --> Decreasing transformation
- Split --> Division / branching
- Flower --> Result of growth
- Leaf --> Result of reduction

6. Grammar (CFG)

6.1 Introduction

This section defines the formal structure and behavior of the proposed nature-inspired symbolic programming language. A Context-Free Grammar (CFG) is used to describe the syntax of the language, while semantics define how expressions are evaluated and interpreted.

6.2 Context-Free Grammar (CFG)

- **Terminals**

Terminals are the atomic symbols that appear literally in source code. They are divided into the following groups:

Terminals	Category
seed - cut - grow - split	Opening entities
Flower - leaf - tree - root	Closing entities

Terminals	Category
Propagate – harvest – bloom – wither – average – sort – uproot – unique - flourish	Built-in functions
+ - * /	Operators
> [] ,	Special symbols
0 1 2 3 4 5 6 7 8 9	Numeric digits

- **Non-terminals**

Non-terminals are intermediate symbols used inside production rules to describe the structure of the language.

Non-terminal	Description
<program>	A single executable statement
<statement>	Either an arithmetic expression or a function call
<arith_expr>	Entity-bracketed repeated-operator expression
<func_call>	Built-in function applied to a list
<entity_open>	One of the four opening entity keywords
<entity_close>	One of the four closing entity keywords
<op_seq>	One or more identical operator symbols
<op>	A single operator symbol
<func_name>	One of the ten standard built-in function names
<arg_list>	Possibly empty comma-separated list of numbers
<args>	Non-empty comma-separated sequence of numbers
<number>	One or more digits forming an integer
<digit>	A single decimal digit (0–9)

- **Production Rules**

The complete set of production rules is listed below. The start symbol is <program>.

```

<program>    → <statement>
<statement>  → <arith_expr>
              | <func_call>
<arith_expr> → <entity_open> <number> <op_seq> <number> <entity_close>
<entity_open> → seed | cut | grow | split
<entity_close> → flower | leaf | tree | root
<op_seq>      → <op>
              | <op> <op_seq>
<op>          → + | - | * | /
<func_call>   → <func_name> [ <arg_list> ]
              | flourish [ <arg_list> ] <number>
<func_name>   → propagate | harvest | bloom | wither
              | average | sort | uproot | unique
<arg_list>    → ε
              | <args>
<args>        → <number>
              | <number> , <args>
<number>      → <digit>
              | <digit> <number>
<digit>       → 0|1|2|3|4|5|6|7|8|9

```

- **Entity paring constraints:**

A well-formedness constraint that cannot be captured by a pure CFG is the matching requirement between opening and closing entities. The parser enforces it explicitly at runtime:

Opening entity	Required closing entity	Operation type
seed	flower	Addition (ADD)
cut	leaf	Subtraction (SUB)
grow	tree	Multiplication (MUL)

Opening entity	Required closing entity	Operation type
split	root	Division (DIV)

- **Some Grammar derivations:**

1. Arithmetic Expression: seed 5 +++> 3 flower

This expression applies addition three times: $5 + 3 + 3 + 3 = 14$.

```

<program>
⇒ <statement> [program → statement]
⇒ <arith_expr> [statement → arith_expr]
⇒ <entity_open> <number> <op_seq> <number> <entity_close>
    [arith_expr rule]
⇒ seed <number> <op_seq> <number> <entity_close> [entity_open → seed]
⇒ seed 5 <op_seq> <number> <entity_close> [number → digit → 5]
⇒ seed 5 <op> <op_seq> <number> <entity_close> [op_seq → op op_seq]
⇒ seed 5 + <op_seq> <number> <entity_close> [op → +]
⇒ seed 5 + <op> <op_seq> <number> <entity_close> [op_seq → op op_seq]
⇒ seed 5 + + <op_seq> <number> <entity_close> [op → +]
⇒ seed 5 + + <op> <number> <entity_close> [op_seq → op]
⇒ seed 5 + + + <number> <entity_close> [op → +]
⇒ seed 5 + + + > 3 <entity_close> [number → digit → 3]
⇒ seed 5 + + + > 3 flower [entity_close → flower]

```

2. Function Call — harvest [1, 2, 3]

This expression returns the sum of all elements: $1 + 2 + 3 = 6$.

```

<program>
⇒ <statement>
⇒ <func_call> [statement → func_call]
⇒ <func_name> [ <arg_list> ] [func_call rule]
⇒ harvest [ <arg_list> ] [func_name → harvest]
⇒ harvest [ <args> ] [arg_list → args]

```

```

⇒ harvest [ <number> , <args> ]          [args → number , args]
⇒ harvest [ 1 , <args> ]                  [number → 1]
⇒ harvest [ 1 , <number> , <args> ]        [args → number , args]
⇒ harvest [ 1 , 2 , <args> ]               [number → 2]
⇒ harvest [ 1 , 2 , <number> ]             [args → number]
⇒ harvest [ 1 , 2 , 3 ]                   [number → 3]

```

7. Lexical Design:

The lexer (also called the tokeniser) is the first stage of interpretation. It reads the raw source string character by character and groups characters into tokens — the smallest meaningful units of the language. The parser never sees the raw text; it works exclusively with the token stream produced by the lexer.

7.1 Token Types

HerbLang defines nine distinct token types:

Token type	Description	Examples
START	Opening entity keyword	Seed- cut- grow - split
END	Closing entity keyword	Flower - leaf - tree - root
FUNC	Built-in function name	Propagate – harvest – bloom – wither – average – sort – uproot – unique - flourish
NUM	Integer literal (one or more digits)	5 10 42 100
OP	Arithmetic operator symbol	+ - * /
ARROW	Direction / pipe symbol	>
LBRACKET	Open square bracket (list start)	[
RBRACKET	Close square bracket (list end)	]
COMMA	Argument separator	,

Whitespace (spaces, tabs, newlines) is consumed and discarded by the lexer. It is never emitted as a token.

7.2 Tokenization Algorithm

The tokeniser uses a single compiled regular expression that matches all legal lexemes in one pass over the input string. The pattern lists longer keywords before shorter ones to avoid partial matches (for example, split must be matched before any single letter). After each match, the lexeme is classified by checking it against lookup sets and emitting the appropriate (type, value) pair:

```
pattern = re.compile(
    r'(propagate|prune|harvest|bloom|wither|absorb|average'
    r'|uproot|unique|flourish|sort'
    r'|flower|split|seed|grow|leaf|tree|root|cut'
    r'|\d+'
    r'|[\+\-\*/]>|[\[\]\,\s+)'
)
for match in pattern.finditer(text):
    word = match.group().strip()
    if not word: continue          # skip whitespace
    elif word in FUNC_NAMES: emit ('FUNC', word)
    elif word in START_WORDS: emit ('START', word)
    elif word in END_WORDS: emit ('END', word)
    elif word.isdigit(): emit ('NUM', int(word))
    elif word in ops: emit ('OP', word)
    elif word == '>': emit ('ARROW', '>')
    elif word == '[': emit ('LBRACKET', '[')
    elif word == ']': emit ('RBRACKET', ']')
    elif word == ',': emit ('COMMA', ',')
```

Any character that does not match the pattern raises a `ValueError` listing the offending character.

7.3 Lexer Examples

seed 5 +++> 3 flower

Position	Lexeme	Token type	Token value
1	seed	START	"seed"
2	5	NUM	5
3	+	OP	"+"
4	+	OP	"+"
5	+	OP	"+"
6	>	ARROW	">"
7	3	NUM	3
8	flower	END	"flower"

8. Parser implementation

The parser is the second stage of interpretation. It consumes the flat token stream produced by the lexer and verifies that the tokens form a syntactically valid sentence according to the grammar. It does not produce a tree data structure with nodes and pointers; instead, it returns a single compact tuple — the Abstract Syntax Tree (AST) — that encodes everything the interpreter needs.

8.1 Design — Recursive Descent

HerbLang uses a top-down, recursive-descent strategy. The parser inspects the type of the first token to decide which grammar branch to take:

```

if first token is FUNC → parse_func_call()
if first token is START → parse_arith_expr()
otherwise             → raise SyntaxError

```

This works without backtracking because the grammar is LL(1): at every decision point, the first unconsumed token uniquely identifies which production rule applies.

8.2 Parsing Arithmetic Expressions

When the first token is START, the parser follows six ordered steps:

1. Consume the START token. Record the opening entity word (e.g. seed) and look up its associated operation type (ADD, SUB, MUL, or DIV) from the START_WORDS map.
2. Consume a NUM token as n1 (the left operand).
3. Count how many consecutive OP tokens matching the expected operator symbol follow. Let this count be k. If k = 0, raise a SyntaxError.
4. Consume the ARROW token (>). If not present, raise a SyntaxError.
5. Consume a NUM token as n2 (the right operand / repetition value).
6. Consume the END token and verify it matches the paired closing entity for the opener (e.g. seed must be closed by flower). If mismatched, raise a descriptive SyntaxError.

The result of a successful arithmetic parse is the four-tuple:

```
(op, n1, k, n2)

op  — one of 'ADD', 'SUB', 'MUL', 'DIV'
n1  — integer: the initial / left value
k   — integer: how many times to apply the operation
n2  — integer: the right operand applied k times
```

8.3 Parsing Function Calls

When the first token is FUNC, the parser follows these steps:

7. Consume the FUNC token. Record the function name.
8. Consume LBRACKET. If not present, raise a SyntaxError.
9. Collect NUM tokens separated by COMMA tokens until RBRACKET is encountered, building the argument list. An empty list is valid.
10. Consume RBRACKET.
11. If the function name is flourish, attempt to consume one more NUM token as the threshold value. If absent, raise a ValueError at evaluation time.

The result of a successful function parse is the four-tuple:

```
('FUNC', name, args, extra)

name  — string: the function name
args  — list of integers: the argument list
extra — integer or None: the threshold for flourish, None otherwise
```

8.4 Error Reporting

The parser raises Python's built-in `SyntaxError` with a human-readable message for each violation. Examples:

Input	Error raised
seed 5 +++> 3 leaf	Entity mismatch: 'seed' must close with 'flower', not 'leaf'.
seed 5 > 3 flower	Expected at least one '+' after seed5
harvest 1, 2, 3	Expected '[' after harvest
cut 10 --> 0 leaf	Cannot divide by zero (caught by interpreter)
grow 4 **> 0 tree	No error — multiply by 0 is legal; result is 0

9. Abstract Syntax Tree (AST):

The Abstract Syntax Tree (AST) is the data structure that bridges the parser and the interpreter. In HerbLang the AST is deliberately minimal: every expression — no matter how complex its source representation — is reduced to a single Python tuple of four elements. This reflects the language's flat, single-expression design.

9.1 AST Node Structure

There are exactly two kinds of AST node:

Arithmetic Node

```
( op, n1, k, n2 )
```

`op` ∈ { 'ADD', 'SUB', 'MUL', 'DIV' }

`n1` : int — starting value

`k` : int — number of repetitions (≥ 1)

`n2` : int — value applied on each repetition

Function Node

```
( 'FUNC', name, args, extra )
```

name : str — built-in function identifier
args : list[int] — list of integer arguments
extra : int | None — threshold value (flourish only)

9.2 AST Examples

Input expression	AST tuple produced
seed 5 +++> 3 flower	('ADD', 5, 3, 3)
seed 5 +> 3 flower	('ADD', 5, 1, 3)
cut 10 --> 2 leaf	('SUB', 10, 2, 2)
grow 2 **> 3 tree	('MUL', 2, 2, 3)
split 12 //> 3 root	('DIV', 12, 2, 3)
harvest [1, 2, 3]	('FUNC', 'harvest', [1,2,3], None)
propagate [4, 5]	('FUNC', 'propagate', [4,5], None)
flourish [5, 2, 8, 1] 4	('FUNC', 'flourish', [5,2,8,1], 4)
sort [3, 1, 4, 1, 5]	('FUNC', 'sort', [3,1,4,1,5], None)
unique [1, 2, 1, 3, 2]	('FUNC', 'unique', [1,2,1,3,2], None)

9.3 Information Preserved in the AST

The AST deliberately discards all lexical surface information that is not needed for evaluation:

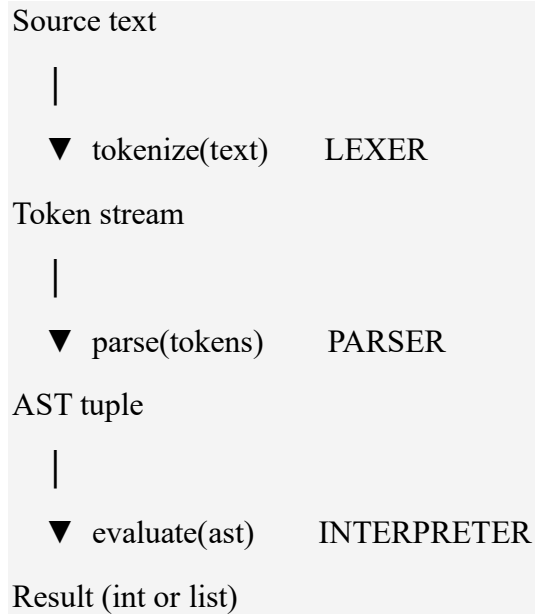
- The entity names (seed, flower, etc.) are discarded after the operation type is established.
- Whitespace is never present in the AST.
- The raw operator characters (+, -, *, /) are replaced by symbolic operation codes (ADD, SUB, MUL, DIV).
- The repetition count k is a single integer, not a list of operator tokens.

This simplification means the interpreter only needs to handle two tuple shapes, making evaluation code short and easy to reason about.

10. Interpreter:

The interpreter is the third and final stage of the pipeline. It receives an AST tuple and computes its value. HerbLang's interpreter uses pure recursion, there are no loops anywhere in the evaluation logic, which is consistent with the functional programming paradigm adopted by the language.

10.1 Pipeline Overview



The entry point `run(code)` chains all three stages: `run = evaluate ◦ parse ◦ tokenize`.

10.2 Evaluating Arithmetic Nodes

Arithmetic evaluation delegates to one of four tail-recursive helper functions depending on the operation code stored in the AST. Each function decrements a counter `k` on every recursive call and returns the accumulated result when `k` reaches zero.

```
def add_repeat(start, k, n2):
    if k == 0: return start
    return add_repeat(start + n2, k - 1, n2)

def sub_repeat(start, k, n2):
    if k == 0: return start
    return sub_repeat(start - n2, k - 1, n2)
```



```
def mul_repeat(start, k, n2):
    if k == 0: return start
    return mul_repeat(start * n2, k - 1, n2)

def div_repeat(start, k, n2):
    if k == 0: return start
    if n2 == 0: raise ZeroDivisionError('Cannot divide by zero')
    return div_repeat(start / n2, k - 1, n2)
```

A dispatch dictionary maps operation codes to their corresponding helper:

```
ops = {
    'ADD': add_repeat,
    'SUB': sub_repeat,
    'MUL': mul_repeat,
    'DIV': div_repeat,
}

result = ops[op](n1, k, n2)
```

When the final result is a float with no fractional part (e.g. 4.0), it is cast back to int before being returned, preserving clean output.

10.3 Worked Arithmetic Evaluations

Expression	AST	Recursive trace	Result
seed 5 +++> 3 flower	('ADD',5,3,3)	add(5,3,3)→add(8,2,3)→ add(11,1,3)→add(14,0,3) →14	14
cut 10 --> 2 leaf	('SUB',10,2,2)	sub(10,2,2)→sub(8,1,2) →sub(6,0,2)→6	6

Expression	AST	Recursive trace	Result
grow 2 **> 3 tree	('MUL',2,2,3)	mul(2,2,3)→mul(6,1,3)→ mul(18,0,3)→18	18
split 81 //> 3 root	('DIV',81,2,3)	div(81,2,3)→div(27,1,3) →div(9,0,3)→9	9

10.4 Evaluating Function Nodes

Function evaluation is handled through a dispatch table that maps each function name to a Python lambda. The lambda receives the argument list and the optional extra value, and calls the appropriate implementation function.

```
FUNCTIONS = {
    'propagate': lambda lst, _: propagate(lst),
    'harvest': lambda lst, _: harvest(lst),
    'bloom': lambda lst, _: bloom(lst),
    'wither': lambda lst, _: wither(lst),
    'average': lambda lst, _: average(lst),
    'sort': lambda lst, _: sort_asc(lst),
    'uproot': lambda lst, _: uproot(lst),
    'unique': lambda lst, _: unique(lst),
    'flourish': lambda lst, extra: flourish(lst, extra),
}
```

The underscore (`_`) parameter in the standard-function lambdas discards the unused extra argument, keeping all entries in the table structurally uniform.

10.5 Worked Function Evaluations

Expression	Operation	Result
harvest [1, 2, 3]	sum([1,2,3])	6
bloom [3, 1, 4, 1, 5]	max([3,1,4,1,5])	5

Expression	Operation	Result
withier [3, 1, 4, 1, 5]	min([3,1,4,1,5])	1
average [2, 4, 6]	sum/len = 12/3	4.0
propagate [1, 2, 3]	lst + lst	[1,2,3,1,2,3]
uproot [1, 2, 3]	lst[::-1]	[3,2,1]
sort [3, 1, 2]	sorted(lst)	[1,2,3]
unique [1, 2, 1, 3, 2]	preserve order, remove duplicates	[1,2,3]
flourish [5,2,8,1] 4	[x for x in lst if x > 4]	[5,8]

11. Built-in- functions

HerbLang provides nine implemented built-in functions, each modelling a natural process applied to a list of numbers. Two additional function names, `prune` and `absorb`, are reserved by the lexer but are not yet implemented in the interpreter; they are described under Future Work.

Function	Syntax	Description	Example input	Output
propagate	propagate [list]	Duplicates the list	[1,2,3]	[1,2,3,1,2,3]
harvest	harvest [list]	Returns the sum of all elements	[1,2,3]	6
bloom	bloom [list]	Returns the maximum element	[3,1,5,2]	5
withier	withier [list]	Returns the minimum element	[3,1,5,2]	1
average	average [list]	Returns the arithmetic mean	[2,4,6]	4.0
sort	sort [list]	Returns the list sorted ascending	[3,1,4,1,5]	[1,1,3,4,5]

Function	Syntax	Description	Example input	Output
uproot	uproot [list]	Returns the list in reverse order	[1,2,3]	[3,2,1]
unique	unique [list]	Removes duplicates, preserving order	[1,2,1,3,2]	[1,2,3]
flourish	flourish [list] N	Filters elements strictly greater than N	[5,2,8,1] 4	[5,8]

12. Extended Examples

12.1 Arithmetic Expressions

Expression	Operation	Calculation	Output
seed 5 +> 3 flower	Add once	$5 + 3$	8
seed 5 ++> 3 flower	Add twice	$5 + 3 + 3$	11
seed 5 +++> 3 flower	Add three times	$5 + 3 + 3 + 3$	14
cut 10 -> 2 leaf	Subtract once	$10 - 2$	8
cut 10 --> 2 leaf	Subtract twice	$10 - 2 - 2$	6
cut 10 ---> 2 leaf	Subtract three times	$10 - 2 - 2 - 2$	4
grow 2 *> 4 tree	Multiply once	2×4	8
grow 2 **> 4 tree	Multiply twice	$2 \times 4 \times 4$	32
grow 2 ***> 4 tree	Multiply 3×	$2 \times 4 \times 4 \times 4$	128
split 81 //> 3 root	Divide twice	$81 \div 3 \div 3$	9
split 100 //// > 10 root	Divide 4×	$100 \div 10 \div 10 \div 10 \div 10$	0

12.2 Function Calls

Expression	Output
propagate [1, 2, 3]	[1, 2, 3, 1, 2, 3]
harvest [10, 20, 30]	60
bloom [7, 2, 9, 4]	9
wither [7, 2, 9, 4]	2
average [10, 20, 30]	20.0
sort [5, 3, 8, 1, 4]	[1, 3, 4, 5, 8]
uproot [1, 2, 3, 4, 5]	[5, 4, 3, 2, 1]
unique [1, 1, 2, 3, 2, 4]	[1, 2, 3, 4]
flourish [1, 5, 3, 9, 2] 3	[5, 9]
flourish [1, 5, 3, 9, 2] 0	[1, 5, 3, 9, 2]

13. Limitations

While the language successfully demonstrates the core concepts of lexical analysis, parsing, and interpretation with a nature-based theme, several limitations exist.

1. **Limited Arithmetic Operations:** The language supports only addition, subtraction, multiplication, and division. It does not support exponentiation, modulo, or parentheses for grouping.
2. **No Variable Storage:** The language has no variables. Each expression is evaluated independently, and results cannot be stored or reused.
3. **No Control Flow:** The language lacks conditionals (if/else) or loops (while/for).
4. **Single Data Type:** The only data type supported is numbers. No strings, booleans, or lists.
5. **Error Handling:** Error messages are minimal. No line numbers, column positions, or helpful suggestions.
6. **No Functions:** The language does not support user defined functions.
7. **REPL Only:** The language only works interactively. It cannot read files.

13.1 Future Work

1. Add variable assignment and lookup
2. Support parentheses and operator precedence
3. User-defined functions

4. Additional data types (strings, Booleans, lists)
5. Better error reporting with line numbers
6. File input support
7. Control flow constructs (if, while)